

LAB 9 LẬP TRÌNH MẠNG VỚI HTTP & SOCKET: GỬI DỮ LIỆU BẰNG PHƯƠNG THỨC POST, TẢI & HIỂN THỊ TRANG WEB, WEB BROWSER CƠ BẢN VÀ GIAO TIẾP CLIENT-SERVER BẰNG SOCKET

1 MỤC ĐÍCH.....	1 2
YÊU CẦU.....	2 3
HƯỚNG DẪN.....	2
3.1 CHUẨN BỊ DỰ ÁN	2
3.2 BÀI 1: CHƯƠNG TRÌNH GỬI DỮ LIỆU BẰNG PHƯƠNG THỨC POST.....	3
3.3 BÀI 2: CHƯƠNG TRÌNH DOWNLOAD VÀ HIỂN THỊ NỘI DUNG TRANG WEB.....	5
3.4 BÀI 3: CHƯƠNG TRÌNH WEB BROWSER CƠ BẢN	7
3.5 BÀI 4: CHƯƠNG TRÌNH GIAO TIẾP CLIENT - SERVER BẰNG SOCKET (TCP)	11
4 CHẠY CHƯƠNG TRÌNH.....	17 5
MỞ RỘNG CHƯƠNG TRÌNH.....	18
5.1 BÀI 5 (NÂNG CAO): TRUYỀN/NHẬN TẬP TIN (FILE TRANSFER) QUA SOCKET KÈM THANH TIỀN TRÌNH 18	
5.2 BÀI 6 (NÂNG CAO): XÂY DỰNG MINI WEB SERVER BẰNG SOCKET (TỰ XỬ LÝ GIAO THỨC HTTP) 24	

1 Mục đích

- Giúp sinh viên hiểu và vận dụng được lớp **HttpClient** trong .NET để giao tiếp với máy chủ Web theo giao thức HTTP.
- Biết cách gửi dữ liệu lên máy chủ bằng phương thức **POST** (dạng form và dạng JSON).

- Biết cách tải (download) nội dung một trang Web bất kỳ về máy, lưu thành tập tin HTML và hiển thị nội dung đó lên giao diện chương trình.
- Xây dựng được một chương trình mô phỏng trình duyệt Web (Web Browser) cơ bản, có khả năng xem và tải toàn bộ tài nguyên của một trang Web (HTML, hình ảnh, CSS, JavaScript).
- Hiểu và vận dụng được lập trình **Socket (TCP)** với các lớp **TcpListener**, **TcpClient**, **NetworkStream** để xây dựng chương trình giao tiếp Client - Server, làm nền tảng cho việc hiểu bản chất hoạt động bên dưới của giao thức HTTP.

2 Yêu cầu

1. Sử dụng **.NET 9.0** trở lên, ngôn ngữ **C#**, ứng dụng **Windows Forms App (.NET)**.
2. Sử dụng lớp **HttpClient** (namespace **System.Net.Http**) để thực hiện các yêu cầu HTTP GET/POST.
3. Chương trình 1: Gửi dữ liệu bất kỳ lên máy chủ bằng phương thức POST và hiển thị kết quả phản hồi.
4. Chương trình 2: Tải nội dung một trang Web từ một địa chỉ URL bất kỳ, lưu thành tập tin .html, sau đó hiển thị nội dung trang Web đó lên Form.
5. Chương trình 3: Xây dựng ứng dụng hoạt động như một Web Browser cơ bản, cho phép xem nội dung Website và tải (download) toàn bộ nội dung trang Web (HTML, hình ảnh, JavaScript, CSS,...) về máy.
6. Chương trình 4: Sử dụng **Socket (TCP)** xây dựng cặp chương trình **Server** và **Client** giao tiếp trực tiếp với nhau qua địa chỉ IP và cổng (port): Server có khả năng lắng nghe, tiếp nhận nhiều Client cùng lúc; Client kết nối, gửi và nhận tin nhắn theo thời gian thực (mô hình chat cơ bản).

3 Hướng dẫn

3.1 Chuẩn bị dự án

1. Mở **Visual Studio 2022** (hoặc mới hơn, có hỗ trợ .NET 9).

Bài Lab môn Lập Trình Mạng Trang **2/28**

Trường Đại học Công Thương TP. Hồ Chí Minh Khoa Công nghệ Thông tin

2. Chọn **Create a new project**, tìm kiếm "Windows Forms App" và chọn C#.
3. Đặt **Framework** là **.NET 9.0**, đặt tên dự án (ví dụ: HttpPostDemo, WebPageDownloader, SimpleWebBrowser, SocketChatServer, SocketChatClient tương ứng với từng bài).
4. Riêng với Bài 3 (Web Browser), cần cài đặt thêm gói thư viện **HtmlAgilityPack** để phân tích cấu trúc HTML: chuột phải vào dự án → **Manage NuGet Packages** → tìm **HtmlAgilityPack** → **Install**. Có thể cài qua Package Manager Console:

Install Install - Package HtmlAgilityPack **Package HtmlAgilityPack**

Nếu dùng .NET CLI: dotnet add package HtmlAgilityPack

1. Kiểm tra tập tin .csproj có khai báo sử dụng Windows Forms như sau:

```
<Project Sdk="Microsoft.NET.Sdk"> <Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup> <PropertyGroup>
    <OutputType>WinExe</OutputType> <OutputType>WinExe</OutputType>
    <TargetFramework>net9.0 <TargetFramework>net9.0 - windows</TargetFramework>
  windows</TargetFramework>
    <Nullable>enable</Nullable> <Nullable>enable</Nullable>
    <UseWindowsForms>true</UseWindowsForms>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>
</Project>
```

3.2 Bài 1: Chương trình gửi dữ liệu bằng phương thức POST

a) Thiết kế Form

1. Thêm các **TextBox**: txtUrl (nhập địa chỉ URL nhận dữ liệu POST), txtName, txtEmail, txtMessage (dữ liệu mẫu cần gửi đi).
2. Thêm **Button** btnSend, đặt thuộc tính Text là "Gửi POST".

3. Thêm **TextBox** txtResult, đặt Multiline = true, ScrollBars = Vertical để hiển thị nội dung phản hồi từ máy chủ.

b) Viết Code

Sử dụng HttpClient để gửi dữ liệu dạng form (application/x-www-form-urlencoded) lên máy chủ bằng phương thức POST. Có thể kiểm thử với các địa chỉ như <https://httpbin.org/post> hoặc <https://postman-echo.com/post>.

Bài Lab môn Lập Trình Mạng Trang 3/28

Trường Đại học Công Thương TP. Hồ Chí Minh Khoa Công nghệ Thông tin

```
using System;

using System.Collections.Generic;

using System.Net.Http;

using System.Windows.Forms;

namespace HttpPostDemo
{
    public partial class Form1 : Form
    {
        private static
        readonly HttpClient client = new HttpClient();

        public Form1()
        {
            InitializeComponent();
        }

        private
        async void btnSend_Click(object sender, EventArgs e)
        {
            string url = txtUrl.Text.Trim();
            if (string.IsNullOrEmpty(url))
            {
                MessageBox.Show("Vui lòng nhập địa chỉ URL.");
                return;
            }
        }
    }
}
```

```

try
{
    btnSend.Enabled = false;
    txtResult.Clear();

    // Dữ liệu gửi đi
    string key = value (application/x-www-form-urlencoded)

    var data = new
Dictionary<string, string> {
    { "name", txtName.Text },
    { "email", txtEmail.Text },
    { "message", txtMessage.Text }
};

    using var content
= new FormUrlEncodedContent(data);

```

Bài Lab môn Lập Trình Mạng Trang 4/28

Trường Đại học Công Thương TP. Hồ Chí Minh Khoa Công nghệ Thông tin

```

    HttpResponseMessage response = await client.PostAsync(url, content);

    string
    responseBody = await response.Content.ReadAsStringAsync();

    txtResult.AppendText($"Mã trả
    ng thái: {(int)response.StatusCode}
    {response.StatusCode} \r\n");

    txtResult.AppendText("----- NỘ
    i dung phản hồi i ----- \r\n");

    txtResult.AppendText(responseBody);
}
catch (Exception ex)
{
    MessageBox.Show("Lỗi khi g

```

```

        if (id != null)
        {
            finally
            {
                btnSend.Enabled = true;
            }
        }
    }
}

```

Ngoài dạng form, có thể gửi dữ liệu dạng **JSON** (application/json) bằng phương thức mở rộng PostAsJsonAsync (namespace System.Net.Http.Json):

```

var payload = new { name = txtName.Text, email = txtEmail.Text, message =
txtMessage.Text };

HttpResponseMessage response = await client.PostAsJsonAsync(url, payload);

string
responseBody = await response.Content.ReadAsStringAsync();

```

3.3 Bài 2: Chương trình Download và hiển thị nội dung trang

Web a) Thiết kế Form

1. Thêm **TextBox** txtUrl để nhập địa chỉ URL của trang Web cần tải.
2. Thêm **Button** btnDownload, đặt Text là "Tải trang Web".
3. Thêm **Label** lblStatus để hiển thị trạng thái/đường dẫn tập tin đã lưu.
4. Thêm control **WebBrowser** (webBrowser1) từ Toolbox, kéo phủ phần lớn diện tích Form để hiển thị nội dung trang Web sau khi tải về.

Bài Lab môn Lập Trình Mạng Trang **5/28**

Trường Đại học Công Thương TP. Hồ Chí Minh Khoa Công nghệ Thông tin

b) Viết Code

```

using System;

using System.IO;

using System.Net.Http;

```

```

using System.Text;

using System.Windows.Forms;

namespace WebPageDownloader
{
    public partial class Form1 : Form
    {
        private static
readonly HttpClient client = new HttpClient();

        public Form1()
        {
            InitializeComponent();
        }

        private
async void btnDownload_Click(object sender, EventArgs e) {
            string url = txtUrl.Text.Trim();
            if (string.IsNullOrEmpty(url)) return;

            try
            {
                btnDownload.Enabled = false;
                lblStatus.Text = "Đang tải";

                string html = await
client.GetStringAsync(url);

                // Lưu nội dung trang Web thành tệp tin HTML
                string fileName = "downloaded_page.html";
                string savePath =
Path.Combine(Application.StartupPath, fileName);
                await
File.WriteAllTextAsync(savePath, html, Encoding.UTF8);
            }
        }
    }
}

```

```

        lblStatus.Text = $"Đã lưu: {savePath}";
    }
}

```

```

// Hiển thị nội dung HTML và tải lên control WebBrowser trong Form

```

Bài Lab môn Lập Trình Mạng Trang 6/28

Trường Đại học Công Thương TP. Hồ Chí Minh Khoa Công nghệ Thông tin

```

        webBrowser1.DocumentText = html;
    }
    catch (Exception ex)
    {
        MessageBox.Show("Lỗi khi tải trang web: " + ex.Message);
    }
    finally
    {
        btnDownload.Enabled = true;
    }
}
}
}
}

```

Control WebBrowser dùng nhân dạng trang cũ (Internet Explorer); nếu muốn hiển thị chuẩn xác các trang hiện đại (HTML5/CSS3/JS mới), có thể thay bằng control Microsoft.Web.WebView2 (cài gói Microsoft.Web.WebView2, phương thức tương ứng là webView21.NavigateToString(html)).

3.4 Bài 3: Chương trình Web Browser cơ bản

a) Thiết kế Form

1. Thêm **TextBox** txtAddress để nhập địa chỉ trang Web (thanh địa chỉ).
2. Thêm các **Button**: btnGo ("Đi đến"), btnBack ("◀"), btnForward ("▶"), btnRefresh ("Tải lại"), btnDownloadPage ("Tải toàn bộ trang").
3. Thêm **Label** lblStatus để hiển thị trạng thái tải trang.
4. Thêm control **WebBrowser** (webBrowser1) chiếm phần lớn diện tích Form để hiển thị

nội dung Website.

b) Viết Code

Chức năng xem Website sử dụng control WebBrowser để điều hướng (Navigate, GoBack, GoForward). Chức năng tải toàn bộ trang sử dụng HttpClient kết hợp thư viện HtmlAgilityPack để phân tích các thẻ chứa tài nguyên (hình ảnh, CSS, JavaScript) và tải từng tài nguyên về thư mục do người dùng chọn.

Bài Lab môn Lập Trình Mạng Trang 7/28

Trường Đại học Công Thương TP. Hồ Chí Minh Khoa Công nghệ Thông tin

```
using System;

using System.Collections.Generic;

using System.IO;

using System.Net.Http;

using System.Text;

using System.Windows.Forms;

using HtmlAgilityPack;

namespace SimpleWebBrowser
{
    public partial class Form1 : Form
    {
        private static
        readonly HttpClient client = new HttpClient();

        public Form1()
        {
            InitializeComponent();

            webBrowser1.Navigated += WebBrowser1_Navigated;

            // ----- Điều hướng cơ bản -----

        }

        private
        void btnGo_Click(object sender, EventArgs e)
        {
            string url =
```

```

txtAddress.Text.Trim(); 
if
(!url.StartsWith("http")) url = "https://" + url;
webBrowser1.Navigate(url);
}

private
void WebBrowser1_Navigated(object sender,
WebBrowserNavigatedEventArgs e)
{
txtAddress.Text =
e.Url.ToString(); this.Text = "Web Browser
- " + webBrowser1.DocumentTitle;
}

private
void btnBack_Click(object sender, EventArgs e) {
if
(webBrowser1.CanGoBack) webBrowser1.GoBack();
}

```

Bài Lab môn Lập Trình Mạng Trang 8/28

Trường Đại học Công Thương TP. Hồ Chí Minh Khoa Công nghệ Thông tin

```

private
void btnForward_Click(object sender, EventArgs e) {
if
(webBrowser1.CanGoForward) webBrowser1.GoForward();
}

private
void btnRefresh_Click(object sender, EventArgs e) {
webBrowser1.Refresh();
}

```

```

// ----- Tả i toàn bộ nộ
i dung trang: HTML + hình ảnh nh + CSS +
JavaScript -- ---
private
async void btnDownloadPage_Click(object sender, EventArgs e) {
    using var folderDialog = new FolderBrowserDialog();
    if (folderDialog.ShowDialog() != DialogResult.OK) return;

    string baseUrl = txtAddress.Text.Trim();
    string saveFolder = folderDialog.SelectedPath;

    try
    {
        btnDownloadPage.Enabled = false;
        lblStatus.Text = "Đang tải trang web...";

        string html = await client.GetStringAsync(baseUrl);
        var doc = new HtmlDocument();
        doc.LoadHtml(html);

        var baseUrl = new Uri(baseUrl);

        // Danh sách th ở và thuộ c tính
        ch ứa đư ở ng d ẫ n tài nguyên c ầ n t ả i
        var resourceNodes =
        new List<(string XPath, string Attribute)> {
            ("//img[@src]", "src"),
            ("//link[@href]", "href"),
            ("//script[@src]", "src")
        };
    }

```

```
foreach (var (xpath, attribute) in resourceNodes)
```

Bài Lab môn Lập Trình Mạng Trang 9/28

Trường Đại học Công Thương TP. Hồ Chí Minh Khoa Công nghệ Thông tin

```
{
    var nodes =
doc.DocumentNode.SelectNodes(xpath);
    if (nodes == null) continue;

    foreach (var node in nodes)
    {
        string resUrl =
node.GetAttributeValue(attribute, "");
        if (string.IsNullOrEmpty(resUrl)) continue;

        if (!Uri.TryCreate(baseUrl, resUrl,
out Uri? fullUri)) continue;

        string localName =
Path.GetFileName(fullUri.LocalPath);
        if (string.IsNullOrEmpty(localName)) continue;

        try
        {
            byte[] data = await
client.GetByteArrayAsync(fullUri);

            string savePath =
Path.Combine(saveFolder, localName);

            await File.WriteAllBytesAsync(savePath, data);

            // C
            p nh
            t l
            ại đư
            ở
            ng
            d ã
            n trong HTML thành t
            p tin c
            ụ
            c
            b
            ộ
        }
    }
}
```

```

node.SetAttributeValue(attribute,
localName); }
catch
{
// Bỏ qua tài nguyên lỗi
tiếp tục xử lý các tài nguyên còn lại
}
}
}

string
htmlPath = Path.Combine(saveFolder, "index.html");
await
File.WriteAllTextAsync(htmlPath, doc.DocumentNode.OuterHtml, Encoding.UTF8);

lblStatus.Text = $"Hoàn tất! Đã lưu trang web vào:
{saveFolder}";
catch (Exception ex)
{

```

Bài Lab môn Lập Trình Mạng Trang 10/28

Trường Đại học Công Thương TP. Hồ Chí Minh Khoa Công nghệ Thông tin

```

MessageBox.Show("Lỗi khi t
ả trang: " + ex.Message);
finally
{
btnDownloadPage.Enabled = true;
}
}
}
}

```

3.5 Bài 4: Chương trình giao tiếp Client - Server bằng Socket (TCP)

Bài này xây dựng **hai dự án Windows Forms riêng biệt**: một chương trình **Server** (lắng nghe và phục vụ nhiều Client) và một chương trình **Client** (kết nối đến Server). Cả hai giao tiếp trực tiếp qua giao thức **TCP** bằng các lớp **TcpListener**, **TcpClient** và **NetworkStream** trong namespace **System.Net.Sockets**, không thông qua HTTP.

a) Thiết kế Form Server

1. Thêm **TextBox** txtPort (nhập số cổng lắng nghe, ví dụ 5000).
2. Thêm **Button** btnStart ("Bắt đầu lắng nghe") và btnStop ("Dừng").
3. Thêm **ListBox** lstClients để hiển thị danh sách Client đang kết nối.
4. Thêm **TextBox** txtLog, đặt Multiline = true, ScrollBars = Vertical để ghi nhận nhật ký (log) hoạt động và tin nhắn nhận được từ các Client.

a) Thiết kế Form Client

1. Thêm hai **TextBox**: txtServerIP (địa chỉ IP máy chủ, ví dụ 127.0.0.1) và txtServerPort (cổng, ví dụ 5000).
2. Thêm **Button** btnConnect ("Kết nối").
3. Thêm **TextBox** txtMessage và **Button** btnSend ("Gửi") để soạn và gửi tin nhắn.
4. Thêm **TextBox** txtChat, đặt Multiline = true, ScrollBars = Vertical để hiển thị hội thoại (tin nhắn gửi/nhận).

b) Viết Code - Chương trình Server

Bài Lab môn Lập Trình Mạng Trang **11/28**

Trường Đại học Công Thương TP. Hồ Chí Minh Khoa Công nghệ Thông tin

Server dùng TcpListener để lắng nghe kết nối trên một cổng cố định. Với mỗi Client kết nối vào, Server tạo một Task riêng để lắng nghe dữ liệu từ Client đó (xử lý đồng thời nhiều Client) và phát (broadcast) tin nhắn nhận được đến tất cả Client còn lại.

```
using System;
using System.Collections.Concurrent;
using System.Net;
using System.Net.Sockets;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace SocketChatServer
{
```

```

public partial class Form1 : Form
{
    private TcpListener? listener;
    private bool isRunning = false;
    private
        readonly ConcurrentDictionary<string, TcpClient> clients = new();

    public Form1()
    {
        InitializeComponent();
    }

    private
        async void btnStart_Click(object sender, EventArgs e)
        {
            int port = int.Parse(txtPort.Text.Trim());
            listener = new
                TcpListener(IPAddress.Any, port);
            listener.Start();
            isRunning = true;
            btnStart.Enabled = false;
            btnStop.Enabled = true;
            Log($"Server đang l ắ ng nghe t ặ c ở
                ng {port}...");

            while (isRunning)
            {
                try
                {
                    TcpClient
                        client = await listener.AcceptTcpClientAsync();

```

Bài Lab môn Lập Trình Mạng Trang 12/28

Trường Đại học Công Thương TP. Hồ Chí Minh Khoa Công nghệ Thông tin

```

        string clientId =
            client.Client.RemoteEndPoint?.ToString() ?? Guid.NewGuid().ToString();
        clients[clientId] = client;

```

```

        Invoke() =>
lstClients.Items.Add(clientId));
        Log($"[{clientId}] đã k ể t n ố i.");

        _ = Task.Run(() => HandleClient(clientId, client));
    }
    catch (Exception)
    {
        // listener đã Stop()
        - > thoát vòng l ặp
        break;
    }
}

private
async Task HandleClient(string clientId, TcpClient client) {
    NetworkStream stream =
client.GetStream();
    byte[] buffer = new byte[4096];

    try
    {
        while (true)
        {
            int
bytesRead = await stream.ReadAsync(buffer);
            if (bytesRead == 0) break; // Client
đã đóng k ể t n ố i

            string message =
Encoding.UTF8.GetString(buffer, 0, bytesRead);
            Log($"[{clientId}]: {message}");

            // Phát (broadcast) tin nh ắn đ ể
            n các Client còn l ại

```

Bài Lab môn Lập Trình Mạng Trang **13/28**
 Trường Đại học Công Thương TP. Hồ Chí Minh Khoa Công nghệ Thông tin

Bài Lab môn Lập Trình Mạng Trang **13/28**
 Trường Đại học Công Thương TP. Hồ Chí Minh Khoa Công nghệ Thông tin

```

        }
    }

    private
    void btnStop_Click(object sender, EventArgs e) {
        this.isRunning = false;
        this.listener?.Stop();
        this

        foreach (var kv
in clients) kv.Value.Close();
        clients.Clear();
        this.btnStart.Enabled = true;
        this.btnStop.Enabled = false;
        this.Log("Server đã d ù ng.");
    }

    private void Log(string text)
    {
        if (txtLog.InvokeRequired)
            txtLog.Invoke(() =>
txtLog.AppendText(text + " \r \n"));
    }

```

Bài Lab môn Lập Trình Mạng Trang 14/28

Trường Đại học Công Thương TP. Hồ Chí Minh Khoa Công nghệ Thông tin

```

    } else
    {
        txtLog.AppendText(text + " \r \n");
    }
}
}
}

```

b) Viết Code - Chương trình Client

Client dùng TcpClient để kết nối đến Server theo địa chỉ IP và cổng; sau khi kết nối, một vòng lặp chạy nền (Task) liên tục đọc dữ liệu từ NetworkStream để nhận tin nhắn theo thời gian thực, song song với việc người dùng có thể gửi tin nhắn bất kỳ lúc nào.

```

using System;
using System.Net.Sockets;
using System.Text;

```

```

using System.Threading.Tasks;
using System.Windows.Forms;

namespace SocketChatClient
{
    public partial class Form1 : Form
    {
        private TcpClient? client;
        private NetworkStream? stream;

        public Form1()
        {
            InitializeComponent();
        }

        private
        async void btnConnect_Click(object sender, EventArgs e)
        {
            try
            {
                string ip = txtServerIP.Text.Trim();
                int port =
                int.Parse(txtServerPort.Text.Trim());

                client = new TcpClient();
                await client.ConnectAsync(ip, port);
                stream = client.GetStream();
            }
        }
    }
}

```

Bài Lab môn Lập Trình Mạng Trang 15/28

Trường Đại học Công Thương TP. Hồ Chí Minh Khoa Công nghệ Thông tin

```

        btnConnect.Enabled = false;
        btnSend.Enabled = true;
        AppendChat("Đã kết nối tới Server.");

        _ = Task.Run(ReceiveLoopAsync);
    }

    catch (Exception ex)
    {
    }
}

```

```

        {
            MessageBox.Show("Không thể
            kết nối: " + ex.Message);
        }
    }

```

```

    private async Task ReceiveLoopAsync()
    {
        byte[] buffer = new byte[4096];
        try
        {
            while (true)
            {
                int
                bytesRead = await stream!.ReadAsync(buffer);
                if (bytesRead == 0) break; // Server

```

đã đóng kết nối

```

            {
                string message =
                Encoding.UTF8.GetString(buffer, 0, bytesRead);
                AppendChat(message);
            }
        }
        catch (Exception ex)
        {
            AppendChat("Mất kết
            nối: " + ex.Message);
        }
    }
}

```

```

    private
    async void btnSend_Click(object sender, EventArgs e)
    {
        string message = txtMessage.Text.Trim();
        if
        (string.IsNullOrEmpty(message) || stream == null) return;

```

```

        byte[] data =

```

```
Encoding.UTF8.GetBytes(message);
await stream.WriteAsync(data);
```

Bài Lab môn Lập Trình Mạng Trang 16/28

Trường Đại học Công Thương TP. Hồ Chí Minh Khoa Công nghệ Thông tin

```
AppendChat("[Tôi]: " + message);
txtMessage.Clear();
}

private void AppendChat(string text)
{
    if (txtChat.InvokeRequired)
    {
        txtChat.Invoke(() => txtChat.AppendText(text + "\r\n"));
    }
    else
    {
        txtChat.AppendText(text + "\r\n");
    }
}
}
```

Đây là mô hình Socket TCP hướng kết nối (connection-oriented): dữ liệu được đảm bảo đến đúng thứ tự, không mất mát. Có thể thử nghiệm bằng cách chạy 1 Server và nhiều Client (mở nhiều lần chương trình Client) trên cùng máy với địa chỉ 127.0.0.1, hoặc trên các máy khác nhau trong cùng mạng LAN bằng địa chỉ IP thực của máy chạy Server.

4 Chạy Chương Trình

- Bài 1:** Nhập địa chỉ (ví dụ <https://httpbin.org/post>), nhập dữ liệu mẫu, nhấn "Gửi POST" và quan sát nội dung phản hồi JSON trả về ở khung kết quả.
- Bài 2:** Nhập địa chỉ một trang Web bất kỳ (ví dụ <https://example.com>), nhấn "Tải trang Web", kiểm tra tập tin `downloaded_page.html` được tạo ra và nội dung hiển thị trên Form.
- Bài 3:** Nhập địa chỉ Website vào thanh địa chỉ, nhấn "Đi đến" để duyệt Web; dùng các nút Back/Forward/Tải lại để điều hướng; nhấn "Tải toàn bộ trang", chọn thư mục lưu và

kiểm tra các tập tin HTML, hình ảnh, CSS, JavaScript đã được tải về.

4. **Bài 4:** Chạy chương trình **Server** trước, nhập cổng (ví dụ 5000) và nhấn "Bắt đầu lắng nghe". Sau đó chạy chương trình **Client** (có thể chạy nhiều bản để mô phỏng nhiều người dùng), nhập IP 127.0.0.1 và cổng 5000, nhấn "Kết nối", rồi gửi tin nhắn qua lại để kiểm tra Server phát (broadcast) tin nhắn đến các Client khác và ghi log đầy đủ.

Bài Lab môn Lập Trình Mạng Trang 17/28

Trường Đại học Công Thương TP. Hồ Chí Minh Khoa Công nghệ Thông tin

5 Mở rộng chương trình

- Hiện thị **ProgressBar** thể hiện tiến trình tải trang và tải tài nguyên.
- Hỗ trợ gửi dữ liệu POST dạng **multipart/form-data** để đính kèm tập tin (upload file).
- Thêm chức năng **Lịch sử duyệt Web (History)** và **Trang yêu thích (Bookmark/Favorites)** cho chương trình Web Browser.
- Dùng **async/Task.WhenAll** để tải song song nhiều tài nguyên, giúp tăng tốc độ tải trang.
- Thay control WebBrowser (IE) bằng **Microsoft Edge WebView2** để hiển thị trang Web hiện đại, chính xác hơn.
- Xử lý thêm các **Header** tùy chỉnh (User-Agent, Authorization, Cookie,...) khi gửi yêu cầu HTTP.
- Xây dựng thêm phiên bản **UDP** (dùng UdpClient) cho chương trình Socket để so sánh với TCP về độ tin cậy và tốc độ truyền dữ liệu.
- Bổ sung cơ chế đặt **tên hiển thị (nickname)** cho từng Client và gửi tin nhắn riêng (private message) giữa hai Client cụ thể thông qua Server.
- Mã hoá dữ liệu truyền qua Socket (ví dụ dùng SslStream/TLS) để đảm bảo an toàn thông tin khi giao tiếp qua mạng.

Để rèn luyện sâu hơn, sinh viên thực hiện thêm **hai bài tập nâng cao** dưới đây, kết hợp

và mở rộng độ khó so với các bài trên.

5.1 Bài 5 (Nâng cao): Truyền/nhận tập tin (File Transfer) qua Socket kèm thanh tiến trình

Bài này khó hơn Bài 4 ở chỗ dữ liệu truyền đi là **nhị phân** (không phải chuỗi văn bản đơn giản), có kích thước lớn nên phải đọc/ghi theo từng khối (buffer), đồng thời phải tự thiết kế **giao thức riêng** (tên tệp, kích thước tệp, dữ liệu) và cập nhật **ProgressBar** theo thời gian thực trong lúc truyền.

Bài Lab môn Lập Trình Mạng Trang **18/28**

Trường Đại học Công Thương TP. Hồ Chí Minh Khoa Công nghệ Thông tin

a) Thiết kế Form Server (FileTransferServer)

1. Thêm **TextBox** txtPort (cổng lắng nghe) và **TextBox** txtSaveFolder (thư mục lưu tệp nhận được, có thể chọn bằng FolderBrowserDialog).
2. Thêm **Button** btnStart ("Bắt đầu nhận").
3. Thêm **ProgressBar** progressBar1 để hiển thị tiến trình nhận tệp.
4. Thêm **TextBox** txtLog (Multiline) để ghi nhật ký.

a) Thiết kế Form Client (FileTransferClient)

1. Thêm **TextBox** txtServerIP, txtServerPort và **TextBox** txtFilePath (đường dẫn tệp, chọn qua OpenFileDialog) cùng **Button** btnChooseFile ("Chọn tệp").
2. Thêm **Button** btnSend ("Gửi tệp") và **ProgressBar** progressBar1 để hiển thị tiến trình gửi.

b) Viết Code - Server

Giao thức tự quy ước đơn giản: Client gửi lần lượt **tên tệp**, **kích thước tệp** (8 byte), rồi đến **dữ liệu nhị phân** của tệp. Dùng BinaryReader/BinaryWriter để đọc/ghi các kiểu dữ liệu này trực tiếp trên NetworkStream.

```
using System;  
using System.IO;  
using System.Net;
```

```

using System.Net.Sockets;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace FileTransferServer
{
    public partial class Form1 : Form
    {
        private TcpListener? listener;
        private bool isRunning = false;

        public Form1()
        {

```

Bài Lab môn Lập Trình Mạng Trang 19/28

Trường Đại học Công Thương TP. Hồ Chí Minh Khoa Công nghệ Thông tin

```

InitializeComponent();
    }

    private
    async void btnStart_Click(object sender, EventArgs e)
    {
        int port = int.Parse(txtPort.Text.Trim());

        listener = new
        TcpListener(IPAddress.Any, port);
        listener.Start();

        isRunning = true;
        btnStart.Enabled = false;

        Log($"Server sã n sàng nhậ n tậ p
tin tậ i cở ng {port}...");

        while (isRunning)
        {
            try
            {
                TcpClient client = await
                listener.AcceptTcpClientAsync();
                _ = Task.Run(() => ReceiveFileAsync(client));

```

```

        }
    } catch (Exception) { break; }
    }
}

private
async Task ReceiveFileAsync(TcpClient client) {
    using (client)
    {
        using
        (NetworkStream stream = client.GetStream())
        {
            using (var reader =
            new BinaryReader(stream))
            {
                try
                {
                    // Giao th [tên t] [kích thư
                    // ở c 8 byte] [d] [ữ] [li] [ệ] [u nh] [i] [phân]
                    string fileName =
                    reader.ReadString();
                    long fileSize = reader.ReadInt64();

                    string savePath =
                    Path.Combine(txtSaveFolder.Text, fileName);
                    using var fs = new
                    FileStream(savePath, FileMode.Create, FileAccess.Write);

                    byte[] buffer = new byte[8192];

```

Bài Lab môn Lập Trình Mạng Trang 20/28

Trường Đại học Công Thương TP. Hồ Chí Minh Khoa Công nghệ Thông tin

```

        long totalReceived = 0;

        while (totalReceived < fileSize)
        {
            int toRead =

```

```

(int)Math.Min(buffer.Length, fileSize - totalReceived);
    }

    int bytesRead =
stream.Read(buffer, 0, toRead);
    if (bytesRead == 0) break;

    fs.Write(buffer, 0, bytesRead);
    totalReceived += bytesRead;

    int percent = (int)(totalReceived *
100 / fileSize);
    UpdateProgress(percent);
    }

    Log($"Nhập xong tệp tin: {fileName} ({fileSize:N0} byte).");
    }
    catch (Exception ex)
    {
        Log("Lỗi khi nhập tệp tin: " + ex.Message);
    }
}

private void UpdateProgress(int percent)
{
    if (progressBar1.InvokeRequired)
    {
        progressBar1.Invoke(() => progressBar1.Value = Math.Min(percent, 100));
    }
    else
    {
        progressBar1.Value =
Math.Min(percent, 100);
    }
}

private void Log(string text)
{

```

```

        if (txtLog.InvokeRequired)
        {
            txtLog.Invoke(() => txtLog.AppendText(text + " \r\n"));
        }
        else
        {
            txtLog.AppendText(text + " \r\n");
        }
    }
}

```

Bài Lab môn Lập Trình Mạng Trang 21/28

Trường Đại học Công Thương TP. Hồ Chí Minh Khoa Công nghệ Thông tin

```

    }
}
}

```

b) Viết Code - Client

```

using System;
using System.IO;
using System.Net.Sockets;
using System.Windows.Forms;

namespace FileTransferClient
{
    public partial class Form1 : Form
    {
        private string? selectedFilePath;

        public Form1()
        {
            InitializeComponent();
        }

        private void btnChooseFile_Click(object sender, EventArgs e)
        {
            using var dialog = new OpenFileDialog();
            if (dialog.ShowDialog() == DialogResult.OK)
            {
                selectedFilePath =
            }
        }
    }
}

```

```

dialog.FileName; 
txtFilePath.Text =
selectedFilePath; 
}
}

private
async void btnSend_Click(object sender, EventArgs e) {
    if
(string.IsNullOrEmpty(selectedFilePath)) {
        MessageBox.Show("Vui lòng
chọn tệp tin cần gửi.");
        return;
    }

    string ip = txtServerIP.Text.Trim();

```

Bài Lab môn Lập Trình Mạng Trang 22/28

Trường Đại học Công Thương TP. Hồ Chí Minh Khoa Công nghệ Thông tin

```

int port =
int.Parse(txtServerPort.Text.Trim());

try
{
    btnSend.Enabled = false;
    progressBar1.Value = 0;

    using var client = new TcpClient();
    await
client.ConnectAsync(ip, port);
    using NetworkStream
stream = client.GetStream();
    using var writer = new
BinaryWriter(stream);

    var fileInfo =
new FileInfo(selectedFilePath);
    writer.Write(fileInfo.Name); // tên tệp

```

```
writer.Write(fileInfo.Length); // kích thước nội dung tệp (8 byte)
```

```
using var fs = new  
FileStream(selectedFilePath, FileMode.Open, FileAccess.Read);  
byte[] buffer = new byte[8192];  
long totalSent = 0;  
int bytesRead;  
  
while ((bytesRead =  
fs.Read(buffer, 0, buffer.Length)) > 0) {  
    await  
stream.WriteAsync(buffer, 0, bytesRead);  
    totalSent += bytesRead;  
  
    int percent = (int)(totalSent  
* 100 / fileInfo.Length);  
    progressBar1.Value = Math.Min(percent,  
100);  
  
    MessageBox.Show("Gửi  
thành công!");  
  
    catch (Exception ex)  
    {  
        MessageBox.Show("Lỗi khi g  
ửi tin: " + ex.Message);  
    }  
    finally  
    {  
        btnSend.Enabled = true;
```

Bài Lab môn Lập Trình Mạng Trang 23/28

Trường Đại học Công Thương TP. Hồ Chí Minh Khoa Công nghệ Thông tin

```
}  
}  
}
```

□ }

Chạy thử: mở FileTransferServer, chọn thư mục lưu và nhấn "Bắt đầu nhận"; mở FileTransferClient, chọn một tệp bất kỳ (ảnh, văn bản,...), nhập đúng IP/cổng của Server rồi nhấn "Gửi tệp". Quan sát ProgressBar ở cả hai bên và kiểm tra tệp nhận được tại Server có giống hệt tệp gốc (đối chiếu dung lượng).

5.2 Bài 6 (Nâng cao): Xây dựng Mini Web Server bằng Socket (tự xử lý giao thức HTTP)

Đây là bài khó nhất của cả buổi thực hành vì yêu cầu **kết hợp kiến thức HTTP (Bài 1-3) và Socket (Bài 4-5)**: thay vì dùng HttpClient để gọi đến một máy chủ có sẵn, sinh viên phải tự đóng vai trò máy chủ, dùng Socket ở mức thấp để **tự đọc và dựng thông điệp HTTP** (Request Line, Header, Body) theo đúng cấu trúc giao thức HTTP.

a) Thiết kế Form (MiniHttpServer)

1. Thêm **TextBox** txtPort (cổng chạy Web Server, ví dụ 8080).
2. Thêm **Button** btnStart ("Khởi động Server").
3. Thêm **TextBox** txtLog, đặt Multiline = true, ScrollBars = Vertical để ghi lại các yêu cầu (request) mà Server nhận được.
4. Tạo thư mục **wwwroot** cạnh tập tin thực thi, đặt vào đó một tập tin **index.html** mẫu (và có thể thêm ảnh, CSS) để Server phục vụ.

b) Viết Code

Server lắng nghe bằng TcpListener như Bài 4; nhưng với mỗi kết nối đến, thay vì đọc chuỗi tin nhắn tự do, Server phải đọc đúng **dòng yêu cầu** dạng "GET /index.html HTTP/1.1", bỏ qua các dòng Header, tìm tập tin tương ứng trong thư mục wwwroot, rồi tự dựng và gửi về một **thông điệp phản hồi HTTP hợp lệ** gồm dòng trạng thái (status line), các Header (Content Type, Content-Length) và phần thân (body).

□ using System;

Bài Lab môn Lập Trình Mạng Trang **24/28**

Trường Đại học Công Thương TP. Hồ Chí Minh Khoa Công nghệ Thông tin

□ using System.IO;

```

using System.Net;
using System.Net.Sockets;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace MiniHttpServer
{
    public partial class Form1 : Form
    {
        private TcpListener? listener;
        private bool isRunning = false;

        private
readonly string wwwRoot = Path.Combine(Application.StartupPath, "wwwroot");

        public Form1()
        {
            InitializeComponent();
            Directory.CreateDirectory(wwwRoot);
        }

        private
        async void btnStart_Click(object sender, EventArgs e)
        {
            int port = int.Parse(txtPort.Text.Trim());

            listener = new
            TcpListener(IPAddress.Any, port);
            listener.Start();

            isRunning = true;
            btnStart.Enabled = false;
            Log($"Mini Web Server đang chạy:
            http://localhost:{port}/ (thư mục {wwwRoot})");

            while (isRunning)
            {
                try

```

```

        {
            await
            TcpClient client = await
listener.AcceptTcpClientAsync();
            _ = Task.Run(() => HandleRequestAsync(client));
        }
    } catch (Exception) { break; }

```

Bài Lab môn Lập Trình Mạng Trang 25/28

Trường Đại học Công Thương TP. Hồ Chí Minh Khoa Công nghệ Thông tin

```

    }
}

private
async Task HandleRequestAsync(TcpClient client)
{
    using (client)
    {
        using
        (NetworkStream stream = client.GetStream())
        {
            try
            {
                // Đọc c dòng yêu c ầ
                u: GET /index.html HTTP/1.1
                string
requestLine = await ReadLineAsync(stream);
                if
                (string.IsNullOrEmpty(requestLine)) return;
                Log("Yêu c ầ
                u: " + requestLine);

                string[] parts =
requestLine.Split(' ');
                string method = parts[0];

                string path = parts.Length >
1 ? parts[1] : "/";
                if (path == "/") path = "/index.html";

                // Đ ợ c và b ỏ qua

```

các dòng Header cho để [] n khi g [] ặ [] p dòng tr ở [] [] ng

[] string line;

[]

[] while

(!string.IsNullOrEmpty(line = await
ReadLineAsync(stream))) { }

[] string
filePath = Path.Combine(wwwRoot, path.TrimStart('/'));

[]

[] if (method == "GET" &&

File.Exists(filePath)) [] {

[]

[] byte[] body = await

File.ReadAllBytesAsync(filePath); []

[] await WriteResponseAsync(stream, "200 OK",

[] GetContentType(filePath), body);

[] }

[] else

[] {

[] byte[] body =

Encoding.UTF8.GetBytes("<h1>404 [] - [] Không tìm [] th ấ [] y
trang</h1>");

[]

[] await

WriteResponseAsync(stream, "404 Not Found", [] "text/html; charset=utf

[] - 8", body); []

[] }

Bài Lab môn Lập Trình Mạng Trang 26/28

Trường Đại học Công Thương TP. Hồ Chí Minh Khoa Công nghệ Thông tin

[] }

[] catch (Exception ex)

[] {

[] Log("L ỗi [] i x [] ử [] lý

yêu c ầ [] u: " + ex.Message); []

[] }

[] }

[] }

```

    private
    static async Task WriteResponseAsync(NetworkStream stream, string
        status, string contentType, byte[] body)
    {
        string header =
            $"HTTP/1.1 {status} \r\n" +
            $"Content - Type: {contentType} \r\n" +
            $"Content - Length: {body.Length} \r\n" +
            $"Connection: close \r\n\r\n";

        byte[]
        headerBytes = Encoding.UTF8.GetBytes(header);
        await
        stream.WriteAsync(headerBytes);
        await stream.WriteAsync(body);
    }

    private
    static Task<string> ReadLineAsync(NetworkStream stream) {
        var sb = new StringBuilder();
        int prev = -1, curr;
        while ((curr = stream.ReadByte()) != -1)
        {
            if (prev == '\r' && curr == '\n')
            {
                sb.Length--; // bỏ ký t
                \r đã thêm ở lần đ
                ọc trư
                ở c
                break;
            }
            sb.Append((char)curr);
            prev = curr;
        }
    }

```

```

    }
    return Task.FromResult(sb.ToString());
}

private static string
GetContentType(string filePath)

```

Bài Lab môn Lập Trình Mạng Trang 27/28

Trường Đại học Công Thương TP. Hồ Chí Minh Khoa Công nghệ Thông tin

```

{
    string ext =
    Path.GetExtension(filePath).ToLower(); return
    ext switch
    {
        ".html" or ".htm" => "text/html; charset=utf - 8",
        ".css" => "text/css",
        ".js" => "application/javascript",
        ".png" => "image/png",
        ".jpg" or ".jpeg" => "image/jpeg",
        ".gif" => "image/gif",
        _ => "application/octet - stream"
    };
}

private void Log(string text)
{
    if (txtLog.InvokeRequired)
    {
        txtLog.Invoke(() => txtLog.AppendText(text + " \r \n"));
    }
    else
    {
        txtLog.AppendText(text + " \r \n");
    }
}
}

```

Chạy thử: khởi động MiniHttpServer, sau đó mở trình duyệt bất kỳ (hoặc dùng chính chương trình Web Browser ở Bài 3) và truy cập <http://localhost:8080/> để kiểm tra

trang index.html được phục vụ đúng; thử truy cập một đường dẫn không tồn tại để kiểm tra phản hồi 404. Đây là ví dụ đơn giản hoá nhằm mục đích học tập, chưa xử lý đầy đủ các trường hợp của giao thức HTTP thực tế (giữ kết nối keep-alive, phương thức POST, mã hoá chuyển tiếp,...).